Verification of Physics-Informed Neural Networks for Simple Harmonic Motion: A Pendulum Case Study

Andrew Juliawan

January 25, 2026

Abstract. This project explores the use of Physics-Informed Neural Networks (PINNs) to model the differential equation governing the motion of a simple pendulum under the small-angle approximation. The neural network is trained by embedding the equation in the loss function. The results show that the PINN is capable of learning simple pendulum dynamics and closely matches the analytical solution.

1. Introduction

Many physical systems are described by differential equations derived from fundamental laws such as Newton's laws of motion. These equations are usually solved using analytical methods and numerical solvers. However, in recent years, Physics-Informed Neural Networks (PINNs) have emerged as an alternative approach, where neural networks are trained to satisfy physical laws by incorporating them into the training objective.

The aim of this project is to explore how PINNs work by modeling a simple PINN using the motion of a simple pendulum. This system is chosen because it already has a simple and well-known solution.

2. Physical Model of the Simple Pendulum

Under the small-angle approximation, the equation of motion for a simple pendulum is given by: [1]

$$\frac{d^2\theta}{dt^2} + \frac{g}{L}\theta = 0 \tag{1}$$

where:

- $\theta(t)$is the angular displacement
- g is the acceleration due to gravity ($9.81\frac{m}{s^2}$)
- L is the length of the pendulum (1 meter)

The general solution of equation (1) is: [2]

$$\theta(t) = A\cos(\sqrt{\frac{g}{L}}t) + B\sin(\sqrt{\frac{g}{L}}t) \tag{2}$$

Where A and B are arbitrary constants.

3. Neural Network Architecture

The PINN uses a Multilayer Perceptron (MLP) neural network architecture consisting of:

- One input neuron representing time t

- Two hidden layers each having 32 neurons. Enough to represent a cosine-like function without training becoming slow or unstable.
- Hyperbolic tangent (tanh) activation functions to introduce non-linearity
- One output neuron representing $\theta(t)$

4. Loss Function Formulation

The loss function is defined as: [3]

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{physics}} + \mathcal{L}_{\text{IC}}$$

Where:

- $\mathcal{L}_{physics}$ enforces the pendulum differential equation
- $\mathcal{L}_{IC}$ enforces the initial conditions

4.1. Physics Loss

From the equation (1), we define the residual as:

$$r(t) = \frac{d^2\theta_{NN}}{dt^2}(t) + \frac{g}{L}\theta_{NN}(t)$$

By sampling N collocation points $(t_i)_{i=1}^N$, the mean squared error of the differential equation residual is defined as:

$$\mathcal{L}_{\text{physics}} = \frac{1}{N}\sum_{i=1}^{N}\left[\frac{d^2\theta_{\text{NN}}}{dt^2}(t_i) + \frac{g}{L}\,\theta_{\text{NN}}(t_i)\right]^2$$

4.2. Initial Condition Loss

The initial conditions are:

$$\theta(0) = \theta_0, \qquad \frac{d\theta}{dt}(0) = 0$$

Thus, the initial condition loss formula is defined as:

$$\mathcal{L}_{\text{IC}} = [\theta_{\text{NN}}(0) - \theta_0]^2 + \left[\frac{d\theta_{\text{NN}}}{dt}(0)\right]^2$$

5. Training Procedure

At each training epoch, random time points within the chosen time interval are sampled as collocation points. The loss function is then derived with respect to θ.

The network is trained using the Adam optimizer. In comparison to other optimizers like SGD (Stochastic Gradient Descent), Adam outperforms them all in both training time and convergence accuracy without being too computationally expensive. [4]

The Adam optimizer builds on two key concepts:

1. Momentum

Momentum is used to accelerate the gradient descent process by incorporating an exponentially weighted moving average of past gradients. This helps smooth out the trajectory of the optimization allowing the algorithm to converge faster by reducing oscillations.

The update rule with momentum is:

$$w_{t+1} = w_t - \alpha m_t$$

where:

- m_t is the moving average gradients at time t
- α is the learning rate (default is 0.001)
- w_t and w_{t+1} are the weights at time t and $t+1$ respectively

The momentum term m_{t+1} is updated recursively as:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\frac{\partial L}{\partial w_t}$$

where:

- β_1 is the momentum parameter (typically set to 0.9)
- $\frac{\partial L}{\partial w_t}$ is the gradient loss function with respect to weights at time t

2. RMSprop (Root Mean Square Propagation)

RMSprop is an adaptive learning rate method that uses an exponentially weighted moving average of squared gradients, which helps overcome the problem of diminishing learning rates.

The update rule for RMSprop is:

$$w_{t+1} = w_t - \frac{\alpha_t}{\sqrt{v_t + \epsilon}}\frac{\partial L}{\partial w_t}$$

where:

- v_t is the exponentially weighted average of squared gradients
- ϵ is a small constant (e.g., 10^{-8}) added to prevent division by zero

The two techniques are combined to provide a efficient optimization process for updating weights.

6. Results

After training, the PINN's predicted solution is compared with the analytical solution (2) of the simple pendulum equation in Figure (1). Initially, the PINN's prediction closely matches the analytical solution, which shows that the PINN has learnt the pendulum dynamics.

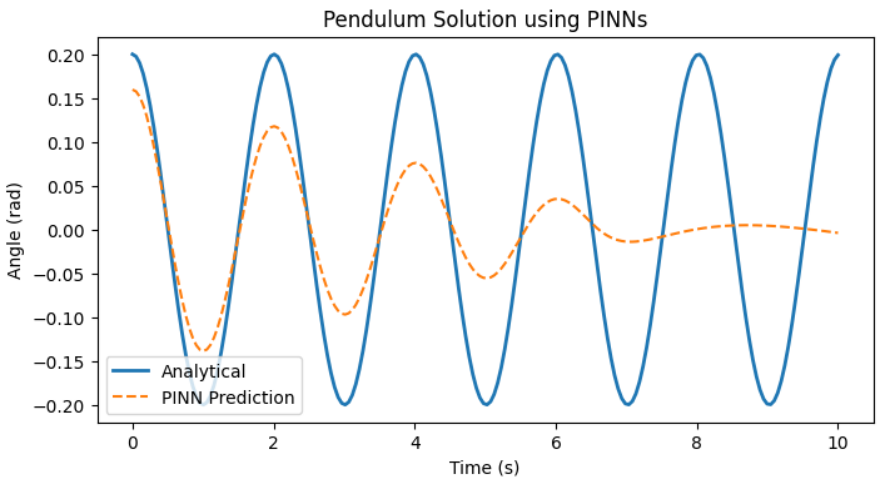

Figure 1: Comparison between analytical solution (2) with PINN prediction.

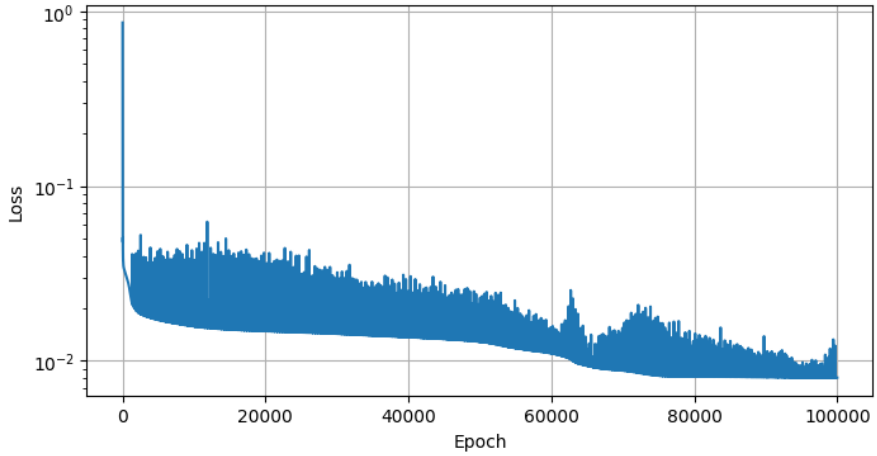

Figure 2: Change in loss value over epoch.

Despite the initial agreement, a closer inspection of Figure (1) reveals that the PINN prediction begins to deviate from the analytical solution after approximately 4 seconds. Specifically, the predicted amplitude (dashed orange line) exhibits a non-physical decay, gradually flattening toward zero.

This phenomenon still needs more investigation, so we can dig deeper and understand the real fundamental factors behind why it happens.

7. Conclusion

This project demonstrates that Physics-Informed Neural Networks can effectively solve the simple pendulum equation. This approach illustrates the potential of PINNs as a powerful tool for solving differential equations and modeling physical systems, especially in scenarios where data is scarce or unavailable. However, the observed decay in the predicted amplitude at later time steps highlights the limitations of standard PINN architectures in modeling long-term periodic motion. Future work could explore strategies to improve temporal stability.

References

[1] David Sang. & Graham Jones. (2021). "Physics Coursebook 3rd Edition". *Cambridge University Press*

[2] Lee Mckelvey. & Martin Crozier. (2018). "Further Mathematics Coursebook". *Cambridge University Press*

[3] Steve Brunton "Physics Informed Machine Learning". *YouTube*. www.youtube.com/playlist?list=PLMrJAkhIeNNQ0BaKuBKY43k4xMo6NSbBa

[4] "What is Adam Optimizer?". *GeeksforGeeks*. 4 October 2025. www.geeksforgeeks.org/deep-learning/adam-optimizer/

[5] S. Juarez. & S. Suraj Subramanian. "Getting Started with PyTorch". *PyTorch*. docs.pytorch.org/tutorials/beginner/basics/intro.html